Buffer Overflow (BOF)

Pratica svolta su Kali Linux

Obiettivo della pratica

In questa esercitazione ho analizzato il funzionamento di un Buffer Overflow (BOF) attraverso un semplice programma in C. Ho creato inizialmente un programma con un buffer di dimensione limitata e una lettura dell'input senza controllo della lunghezza. Successivamente ho provato input più lunghi, ho aumentato la dimensione del buffer e infine ho modificato il codice per limitare e controllare i dati inseriti.

L'obiettivo era osservare il comportamento del programma, capire perché può verificarsi un errore di segmentazione e applicare misure di prevenzione.

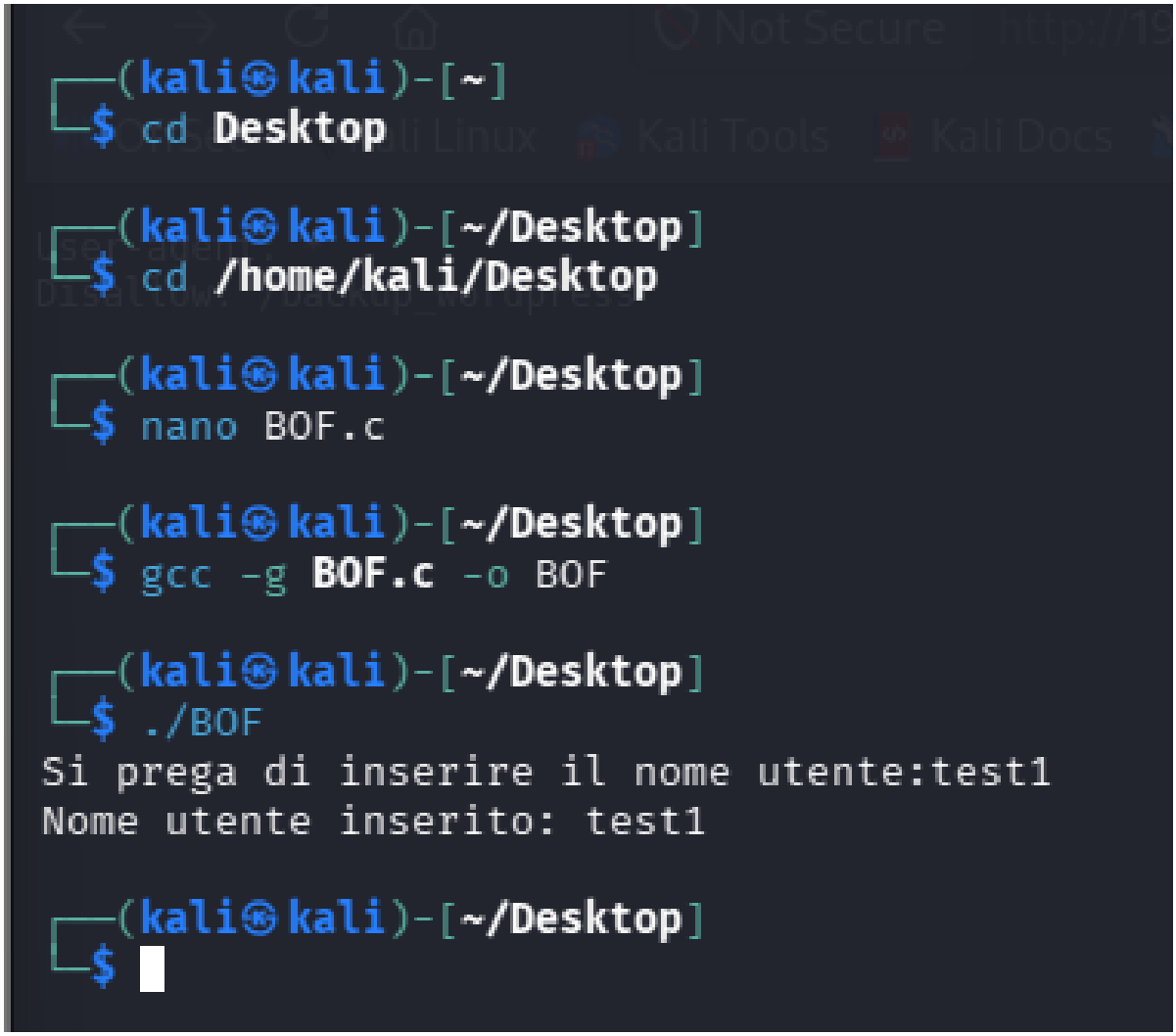
Creazione del programma vulnerabile

Per prima cosa ho aperto il terminale di Kali Linux, mi sono spostato sul Desktop e ho creato il file BOF.c con nano. Il programma iniziale dichiarava un array di 10 caratteri e utilizzava scanf("%s", buffer), che non limita la quantità di caratteri letti.

```
char buffer[10];
scanf("%s", buffer);
```

Primo avvio con input breve

Ho compilato il sorgente con gcc -g BOF.c -o BOF e ho avviato ./BOF. Inserendo il nome breve "test1", il programma ha restituito correttamente il valore inserito.



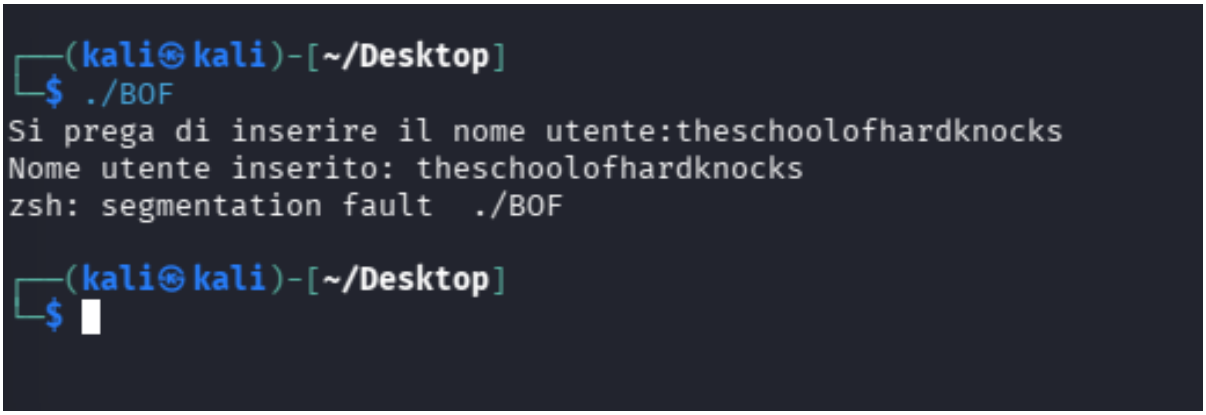
Primo avvio con input breve.

Riproduzione del problema con buffer da 10

Dopo il test con un nome breve, ho ripetuto l'esecuzione inserendo una stringa più lunga dello spazio disponibile nel buffer. Il programma ha stampato il testo e poi la shell ha segnalato "segmentation fault". Questo comportamento è coerente con una scrittura oltre i limiti dell'array: la memoria adiacente può essere sovrascritta e il processo può terminare in modo anomalo.

Errore di segmentazione con buffer da 10

Nello screenshot si vede l'inserimento di una stringa lunga e il messaggio "zsh: segmentation fault ./BOF". È la riproduzione del comportamento vulnerabile.



Errore di segmentazione con buffer da 10.

Aumento del buffer a 30 caratteri

In seguito ho modificato la dichiarazione dell'array da char buffer[10] a char buffer[30] e ho ricompilato il programma. Ho controllato con grep che la modifica fosse effettivamente presente nel sorgente.

Modifica e ricompilazione

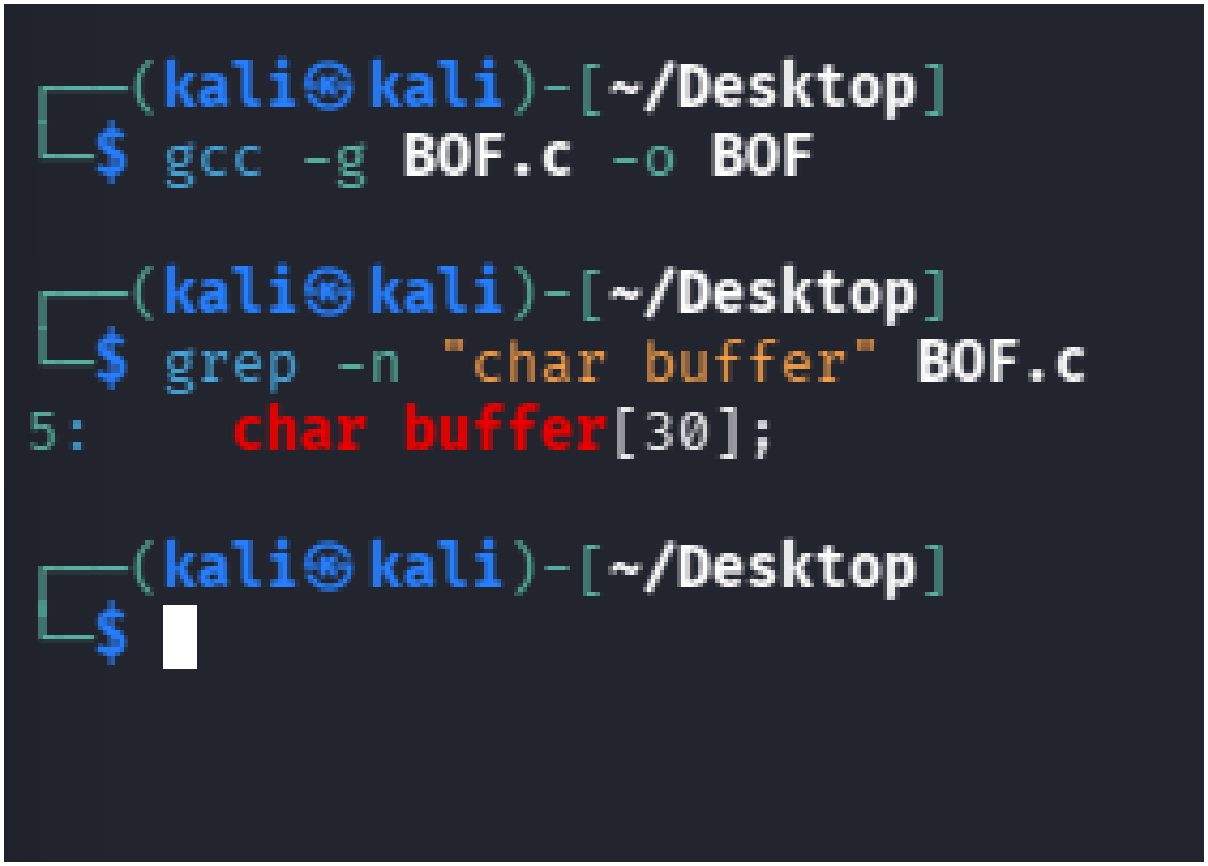
Ho aperto il file BOF.c e ho eseguito nuovamente la compilazione. La compilazione si è conclusa senza messaggi di errore.



Modifica e ricompilazione.

Verifica della dimensione del buffer

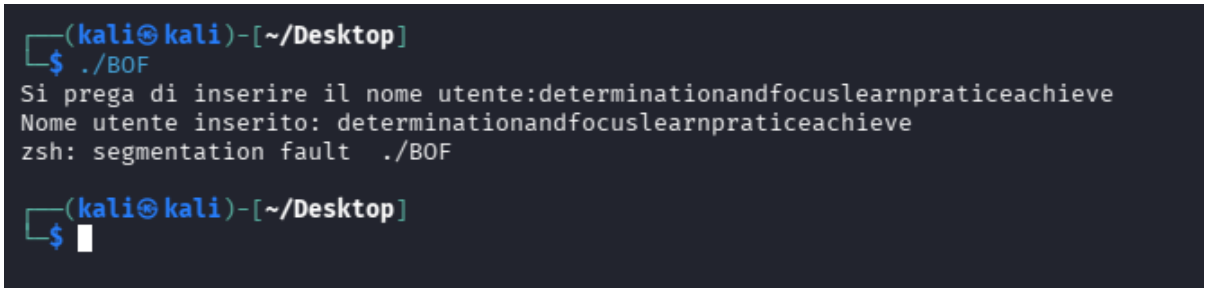
Con grep -n "char buffer" BOF.c ho verificato che il sorgente contenesse la dichiarazione char buffer[30].



Verifica della dimensione del buffer.

Ho ripetuto la prova con una stringa più lunga della capacità del buffer. Anche in questo caso si è verificato un segmentation fault. Questo dimostra che aumentare la dimensione dell'array non risolve da solo il problema: scanf("%s", buffer) continua a non limitare la lunghezza dell'input.

Errore anche con buffer da 30
Lo screenshot mostra che, dopo l'inserimento di una stringa molto lunga, il programma ha nuovamente terminato con segmentation fault.

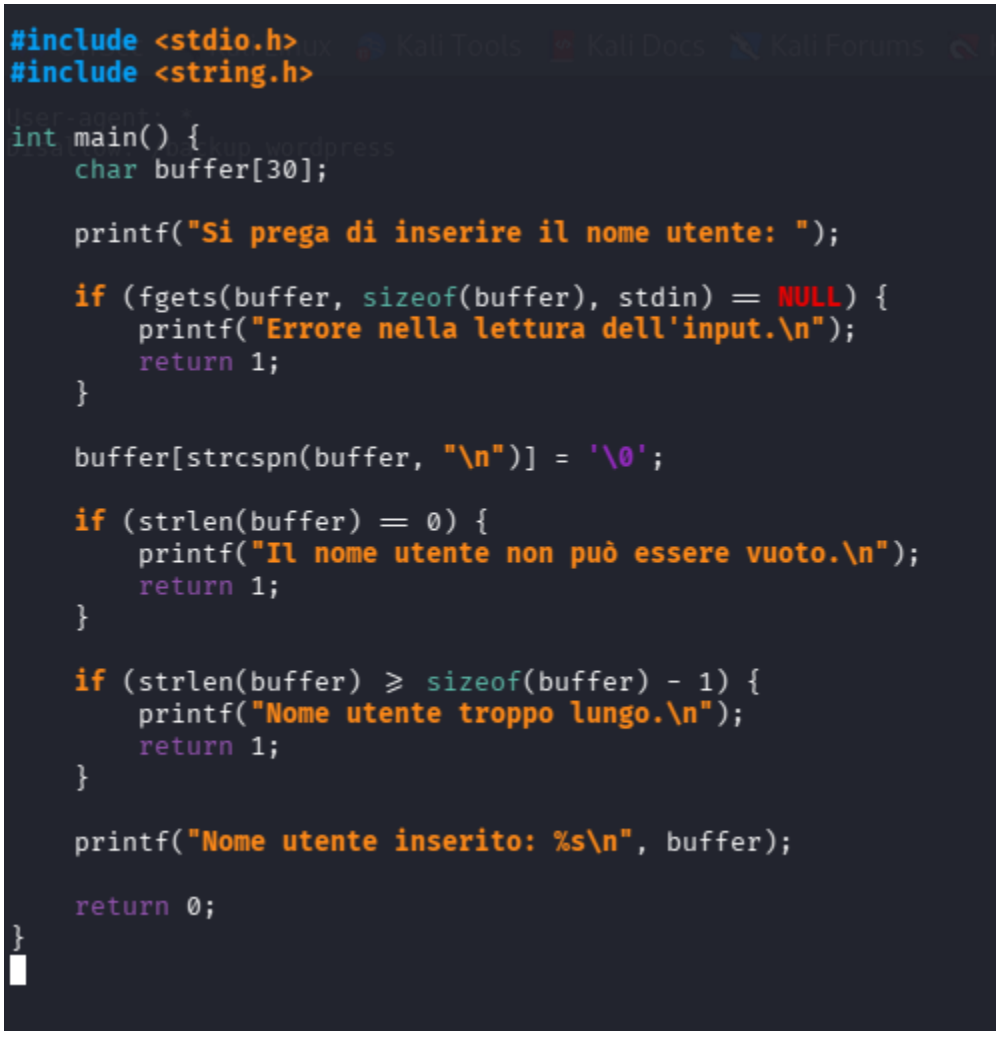


Errore anche con buffer da 30.

Correzione: lettura limitata e controllo dell'input

Per correggere il problema ho sostituito scanf("%s", buffer) con fgets(buffer, sizeof(buffer), stdin). In questo modo la lettura viene limitata alla dimensione del buffer. Ho inoltre rimosso l'eventuale carattere di nuova riga, controllato che il nome non fosse vuoto e rifiutato gli input troppo lunghi.

Ho poi aggiunto una validazione dei caratteri: sono ammessi soltanto lettere maiuscole e minuscole, numeri e underscore. Gli altri caratteri vengono rifiutati.



```c
#include <stdio.h>
#include <string.h>

int main() {
    char buffer[30];

    printf("Si prega di inserire il nome utente: ");

    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        printf("Errore nella lettura dell'input.\n");
        return 1;
    }

    buffer[strcspn(buffer, "\n")] = '\0';

    if (strlen(buffer) == 0) {
        printf("Il nome utente non può essere vuoto.\n");
        return 1;
    }

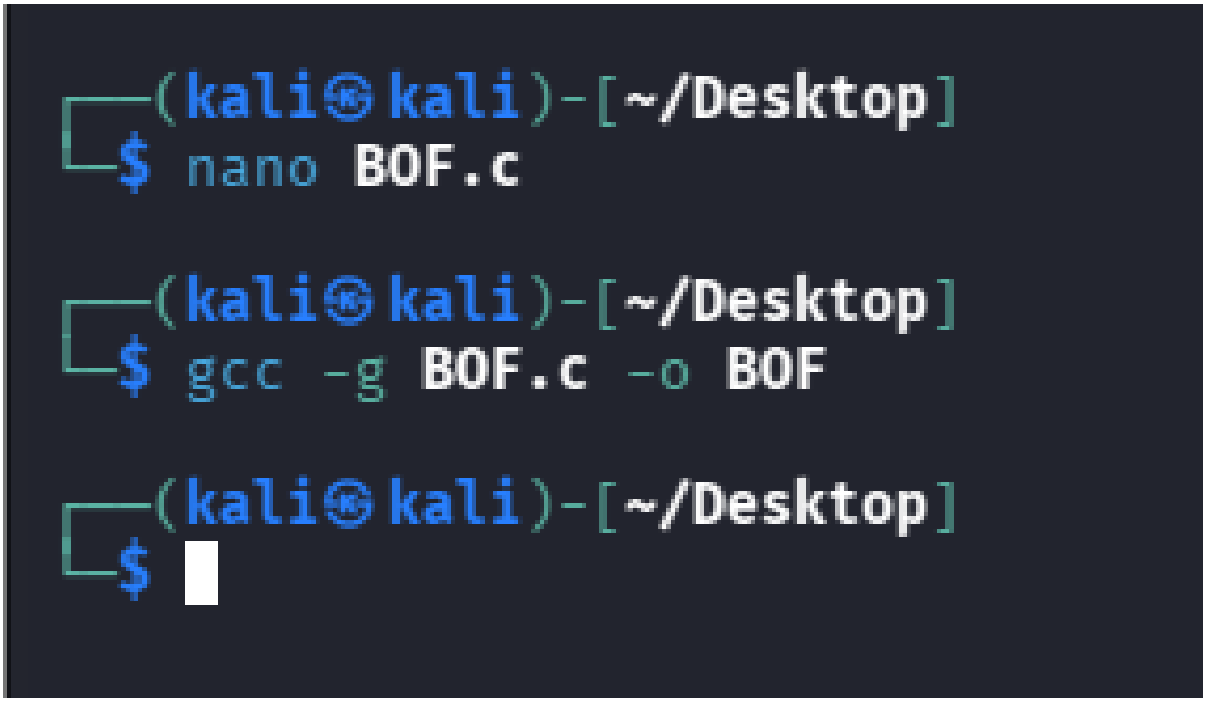
    if (strlen(buffer) >= sizeof(buffer) - 1) {
        printf("Nome utente troppo lungo.\n");
        return 1;
    }

    printf("Nome utente inserito: %s\n", buffer);

    return 0;
}
```

Codice con fgets e controlli

Ho inserito nel file la versione corretta del programma. La schermata mostra l'uso di fgets, il controllo della lunghezza, il controllo del nome vuoto e la validazione dei caratteri ammessi.



Codice con fgets e controlli.

Test della versione corretta

Dopo aver salvato il codice, ho ricompilato il programma ed eseguito i test. Il primo input, “Godfred”, è stato accettato. In un test successivo, “Godfred@95”, il programma ha rifiutato il carattere @. Infine, una stringa molto lunga è stata rifiutata con il messaggio “Nome utente troppo lungo”, senza generare segmentation fault.

Test della versione con controllo della lunghezza

Ho verificato l’accettazione di un nome breve e il rifiuto di una stringa troppo lunga. Il programma ha gestito entrambi i casi senza errore di segmentazione.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ nano BOF.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente: Godfred
Nome utente inserito: Godfred

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente: determinationandfocuslearnpraticeachieve
Nome utente troppo lungo.

┌──(kali㉿kali)-[~/Desktop]
└─$
```

Test della versione con controllo della lunghezza.

Test della sanitizzazione dei caratteri

Ho eseguito tre prove: “Godfred95” è stato accettato; “Godfred@95” è stato rifiutato per la presenza di un carattere non consentito; una stringa molto lunga è stata rifiutata perché supera il limite previsto.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ nano BOF.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Inserisci il nome utente: Godfred95
Nome utente inserito: Godfred95

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Inserisci il nome utente: Godfred@95
Nome utente non valido: caratteri non consentiti.

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Inserisci il nome utente: determinationandfocuslearnpraticeachieve
Nome utente troppo lungo.

┌──(kali㉿kali)-[~/Desktop]
└─$
```

Test della sanitizzazione dei caratteri.

Risposte alle domande della traccia

Aumentare il buffer da 10 a 30 caratteri elimina la possibilità di generare un BOF? No. L'aumento della dimensione offre più spazio, ma non impedisce a un input ancora più lungo di superare il limite. Se la funzione di lettura non limita i dati copiati, il programma resta vulnerabile.

Quali controlli ho aggiunto? Ho utilizzato fgets con la dimensione effettiva del buffer, ho verificato la lunghezza dell'input, ho rifiutato i nomi vuoti e ho consentito soltanto lettere, numeri e underscore. Questi controlli impediscono l'accettazione di input non conformi e limitano la quantità di dati letti.

Conclusioni

Con questa pratica ho osservato concretamente il problema del Buffer Overflow. La prima versione del programma, basata su scanf("%s", buffer), ha prodotto un errore di segmentazione quando ho inserito stringhe più lunghe dello spazio previsto. Ho verificato che anche portando il buffer da 10 a 30 caratteri il problema può ripresentarsi, perché la lettura non era ancora limitata.

Con la versione corretta ho introdotto una lettura limitata tramite fgets e controlli sulla lunghezza e sui caratteri. Nei test documentati il nome valido è stato accettato, mentre gli input troppo lunghi o con caratteri non ammessi sono stati rifiutati. La pratica mi ha mostrato che la prevenzione non consiste soltanto nell'aumentare la dimensione del buffer, ma soprattutto nel gestire e validare correttamente l'input.